



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz

Algorithmen III: Rekursion

Türme von Hanoi

Dezember 2025, Version: 1.22

Fakultät Elektro- und Informationstechnik

1 Einleitung

Viele Probleme lassen sich programmtechnisch sehr elegant durch rekursive Algorithmen lösen.

Das Lexikon sagt dazu:

„Als Rekursion (lateinisch recurrere ‚zurücklaufen‘) bezeichnet man den abstrakten Vorgang, dass Regeln auf ein Produkt, das sie hervorgebracht haben, von neuem angewandt werden. Hierdurch entstehen potenziell unendliche Schleifen. Regeln bzw. Regelsysteme heißen rekursiv, wenn sie die Eigenschaft haben, Rekursion im Prinzip zuzulassen.“
(aus [1])

Im Bereich der Informatik bezeichnet man einen Algorithmus als rekursiv, wenn eine Funktion direkt oder indirekt von sich selbst aufgerufen wird. Da man mit unendlichen Rekursionen in einem endlichen Rechner-System und bei endlicher Verarbeitungszeit nicht viel anfangen kann, kommt hier noch die Abbruchbedingung als wichtiges Element hinzu.

Ein rekursiver Algorithmus besteht also aus:

- Einer Funktion, die direkt oder indirekt von sich selbst aufgerufen wird.
- Einer Abbruchbedingung, welche den weiteren Aufruf der Funktion unterbindet und die Rekursion beendet.

Damit das Ganze funktioniert, muss die Funktion durch die rekursiven Aufrufe das System zuverlässig in Richtung der Abbruchbedingung verschieben, die dann auch irgendwann erfüllt sein muss.

In dieser Aufgabe soll das berühmte Problem der „Türme von Hanoi“ [2] gelöst werden. Dieses Musterproblem für das Arbeiten mit rekursiven Algorithmen kommt sehr oft zum Einsatz, um das Prinzip der Rekursion verständlich und erfahrbar zu machen.

Kurz zusammengefasst geht es darum, einen Stapel Scheiben, die auf einem Stab aufgesteckt sind, eine nach der anderen so auf einen zweiten Stab umzuschichten, dass niemals eine größere auf eine kleinere Scheibe zu liegen kommt.

Hierfür steht neben dem Ausgangsstab, auf dem sich der Stapel zu Beginn befindet, und dem Zielstab, auf dem sich der Stapel am Ende befinden soll, noch ein weiterer Stab zum Zwischenlagern derzeit nicht benötigter Elemente zur Verfügung. In jedem Zug darf nur eine Scheibe bewegt werden.

Weitere Hintergrundinformation finden Sie unter [2].

2 Vorbereitungen

Zur eigentlichen Umsetzung benötigen wir einige Vorbereitungen, die in den nächsten Abschnitten erledigt werden.

Um eine Grafik auf dem Bildschirm ausgeben zu können, benötigt man eine Bibliothek, welche mit dem Betriebssystem des Rechners interagiert und dessen Grafikmöglichkeiten ausschöpfen kann. Im Standard-Sprachumfang der Sprache C ist eine derartige Bibliothek nicht vorgesehen und muss also hinzugefügt werden.

Eine sehr minimalistische Lösung für diese Anforderung stellt das Paket „WinAdapt“, von Franz Kneißl dar, dessen Anleitung [4] Sie im Elearning-Kurs vorfinden.

Um das Paket leichter verwenden und auch gleich testen zu können, finden Sie im Elearning-Kurs ein fertig konfiguriertes Beispielprojekt, das Sie für Ihre Entwicklung als Basis verwenden können.

Hinweis: Das Beispielprogramm ist ein „getarntes“ C++-Programm. Dies muss Sie aber nicht kümmern, solange Ihr Compiler den Standard C++11 beherrscht.

Hinweis: Das Beispielprojekt unterstützt Linux (mit installierten GTK2-Development-Bibliotheken)¹ sowie Windows (keine weiteren Anforderungen).



Entpacken Sie das Beispielprogramm, compilieren und testen Sie es.

3 Die Graphische Darstellung der Türme

Damit das Geschehen später auch auf dem Bildschirm in ansprechender Weise mitverfolgt werden kann, wird eine graphische Darstellung der Spielsituation benötigt. Im Einzelnen bedeutet das, dass die Elemente:

- Stapelbasis mit senkrechtem Stab (drei Stück nebeneinander)
- Scheibenstapel mit konfigurierbarer Anzahl Scheiben unterschiedlicher Größe

dargestellt werden müssen.

Damit das alles auch ordentlich funktioniert, müssen die Größen und Positionen aller Elemente an die Größe des zur Verfügung stehenden Bildschirmfensters angepasst werden. In Abbildung 1 ist eine beispielhafte Darstellung des Ausgangszustands des Spiels dargestellt.

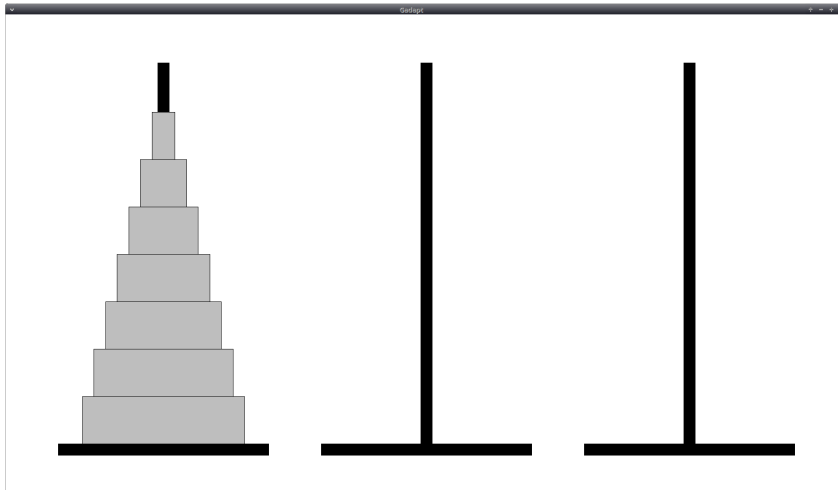


Abbildung 1: Eine Beispieldarstellung für Türme von Hanoi mit sieben Scheiben. Durch die automatische Anpassung der Abmessungen und Positionen an die Fenstergeometrie ergibt sich je nach Fenstergröße und -format bzw. Scheibenanzahl eine im Detail andere Darstellung.

3.1 Datenstruktur zur Verwaltung der Türme

Da das Programm verschiedene Höhen beim Anfangsstapel (Scheibenanzahl bzw. Stockwerke der Türme) unterstützen soll, muss diese Anzahl im Programm konfigurierbar sein. Hierfür kommt eine Präprozessordefinition namens `NUMFLOORS` zum Einsatz. Setzen Sie diese für den Anfang auf eine moderate Zahl, also z.B. im Bereich `[3..7]`.

Nun muss noch für jeden Turm bestimmt werden, wie viele Scheiben welchen Durchmessers auf ihm gestapelt sein sollen. Hierfür bietet es sich an, ein zweidimensionales Array zu verwenden, welches in einer Richtung (erster Index) die Nummer des jeweiligen Turms als Index verwendet, und in der anderen Richtung (zweiter Index) die Etage des jeweiligen Turms.

Damit kann nun in jedem Element des Arrays der Durchmesser der Scheibe in der jeweiligen Etage gespeichert werden. Dabei gehen Sie am besten so vor, dass Sie im Index 0 die Anzahl der gerade vorhandenen Etagen des Turms ablegen und ab dem Index 1 jeweils den Durchmesser der Scheibe in der jeweiligen Etage (oder die Zahl 0).

Als Datenelemente des Turms wählen Sie `int`-Elemente.

Da es sich bei den Türmen von Hanoi stets um drei Türme handelt, welche maximal `NUMFLOORS` Etagen haben, sind nun alle benötigten Informationen zur Definition des Arrays vorhanden.



Definieren Sie eine Präprozessor-konstante zur Festlegung der Anzahl der Scheiben im Spiel.



Definieren Sie eine globale Variable zur Speicherung des derzeitigen Zustands der drei Türme.

¹Die Linux-Variante funktioniert auch auf OSX-Systemen, beachten Sie hierzu die Anleitung im Elearning-Kurs.

Definieren Sie ein globales Array namens `Towers` entsprechend des dargestellten Konzepts und initialisieren Sie alle Elemente des Arrays mit dem Wert 0.

Hinweis: Sie können ein Array (unabhängig von der Größe und der Anzahl seiner Dimensionen) bei der Definition mit einem Wert für alle Elemente initialisieren lassen, indem Sie den Initialisierungswert wie üblich nach einem Gleichheitszeichen aber in geschweiften Klammern angeben.

Das explizite Programmieren einer Schleife zur Initialisierung ist nicht nötig.

Beispiel: `int dummy[42][123][2]={0xff}`

3.2 Darstellung der Stapelbasis



Definieren Sie eine Funktion zum Zeichnen der Stapelbasis.

Beginnen Sie nun mit der Funktion `drawTowers` zum Zeichnen der Türme. Diese benötigt als Parameter das Array `Towers` mit den Turmdaten, sowie die Bildschirmkoordinaten `Xleft`, `Yupper`, `Xright`, `Ylower` (alle vom Typ `int`) wie sie auch der Funktion `VtlPaint` übergeben werden.

Hinweis: Beachten Sie, dass der Koordinatenursprung links oben liegt, also sich die y-Koordinate 0 am oberen Fensterrand befindet.

Bevor die Türme gezeichnet werden können, müssen die Stapelbasen und Stäbe gezeichnet werden. Dabei soll sichergestellt sein, dass das erzeugte Diagramm auch bei unterschiedlichen Größen des Bildschirms ein ansprechendes Aussehen hat.

Daher sollen sich die Abmessungen der Stapelbasen dynamisch mit der Fenstergröße ändern.

Hinweis: Falls Ihnen die Angaben in der folgenden Tabelle nicht klar sind, fertigen Sie sich eine Zeichnung an, in die Sie die relevanten Längen einzeichnen können.

Berechnen Sie zur Vorbereitung des Aufrufs der Zeichenfunktionen bitte zunächst folgende `int`-Größen:

Name	Bedeutung	Berechnungsvorschrift
<code>BaseWidth</code>	Breite der Basisplatte	Breite des Bildschirms geteilt durch vier; Damit wird die gesamte Breite in drei Basisplatten und vier Zwischenräume mit jeweils einem Viertel der Basisplattenbreite eingeteilt.
<code>BaseDistance</code>	Horizontaler Abstand der Basisplatten	Ein Viertel der Breite einer Basisplatte.
<code>PoleHeight</code>	Höhe des Stabs auf der Platte	80 % der Höhe des Bildschirmfensters
<code>DiscUnit</code>	Breiteneinheit der Scheiben	Die größte Scheibe ist <code>NUMFLOORS</code> Einheiten breit und soll um zwei Einheiten kleiner gezeichnet werden als die Basisplatte
<code>PoleWidth</code>	Breite des Stabs	Der Stab soll entweder ein Zehntel der Basisplattenbreite oder die Hälfte einer <code>DiscUnit</code> breit sein, je nachdem was kleiner ist.
<code>DiscHeight</code>	Höhe einer Scheibe	Eine Scheibe soll so dick sein, dass der Stab oben um eine Scheibendicke aus dem Stapel ragt, wenn der Turm mit <code>NUMFLOORS</code> Scheiben komplett gefüllt ist.

Ergänzen Sie nun die Funktion indem Sie die drei Stapelbasen mit den drei Stapelstäben als schwarze gefüllte Rechtecke zeichnen.

Testen Sie nun Ihre Funktion indem Sie sie in der Funktion `VtlPaint` ausführen lassen und verändern Sie die Fenstergröße mit der Maus, um zu verifizieren, dass alle Berechnungen das gewünschte Ergebnis liefern. Korrigieren Sie eventuelle Fehler jetzt.

Hinweis: Bei extrem kleinen Fensterabmessungen von nur wenigen Pixeln Höhe oder Breite kann es sein, dass die Darstellung nicht ganz zufriedenstellend ausfällt. Dies ist in Ordnung und erfordert keine Korrektur.

In Abbildung 3 sind einige Beispiele für korrekte Darstellungen der Basis bei variierter Fenstergeometrie abgebildet.

3.3 Darstellung der Scheibenstapel

Nun müssen die Scheibenstapel noch dargestellt werden. Dafür schreiben Sie zunächst in der Funktion `VtlInit` eine Schleife zur Bestückung des linken Stapels mit insgesamt `NUMFLOORS` Scheiben abnehmender Größe.



Testen Sie Ihre Zeichenfunktion für die Stapelbasen.



Ergänzen Sie die Darstellung um die Scheiben.

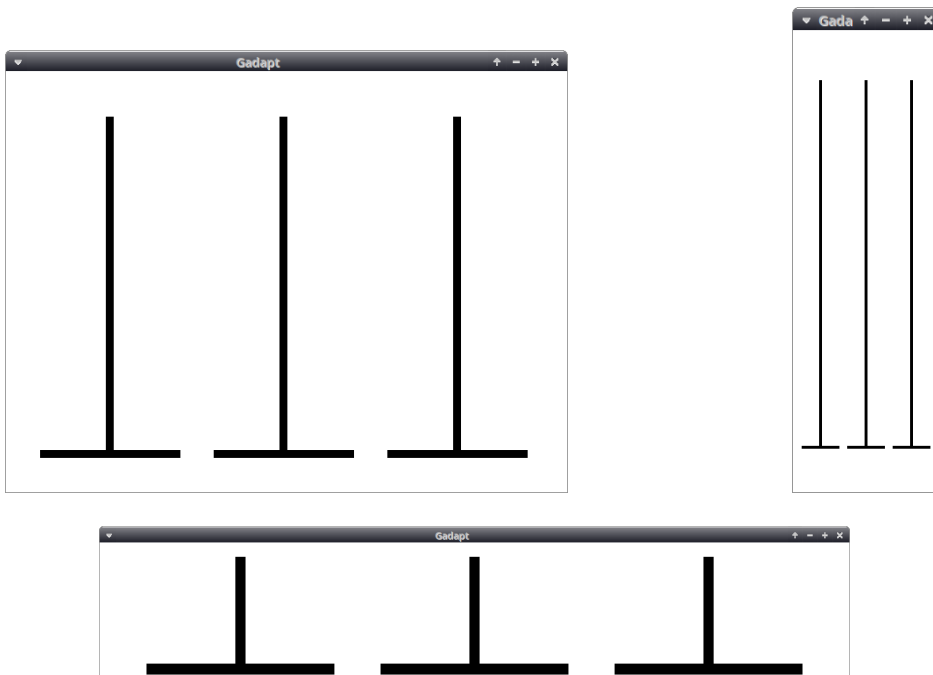


Abbildung 2: Beispiele für unterschiedliche Bildschirmfenstergeometrien.

Die unterste (größte) Scheibe soll dabei im Arrayelement mit dem Etagen-Index 1 platziert werden, die höher liegenden Scheiben dann bei aufsteigenden Indexwerten. Dabei soll die Scheibengröße immer um eins reduziert werden, so dass die oberste Scheibe die Größe eins besitzt.

Das nullte Element des Arrays für jede Basis soll die Anzahl der Scheiben auf der jeweiligen Basis beinhalten.

Damit die Tests aussagekräftig werden, schreiben Sie in den zweiten Stapel den selben Inhalt wie in den ersten Stapel und im dritten Stapel kehren Sie die Reihenfolge der Scheiben um (die kleinste zuunterst).

Hinweis: Vergessen Sie nicht, die Anzahl der Scheiben auf dem Stapel in das 0-te Element des Arrays zu schreiben.

Testen Sie nun Ihre Funktion und verändern Sie die Fenstergröße mit der Maus, um zu verifizieren, dass alle Berechnungen das gewünschte Ergebnis liefern. Korrigieren Sie eventuelle Fehler jetzt.

In Abbildung 3 sind einige Beispiele für korrekte Darstellungen bei variierter Fenstergeometrie abgebildet.

Wenn die Tests zufriedenstellend abgeschlossen sind, entfernen Sie den Inhalt aus dem zweiten und dem dritten Stapel, damit Sie eine Ausgabe entsprechend der Abbildung 1 erhalten.

Hinweis: Vergessen Sie nicht, die Anzahl der Scheiben auf dem Stapel, die im 0-ten Element des Arrays für jeden Stapel gespeichert ist, ebenfalls zu korrigieren.



Testen Sie Ihre komplette Zeichenfunktion.



Entfernen Sie den Inhalt aus dem zweiten und dem dritten Stapel.

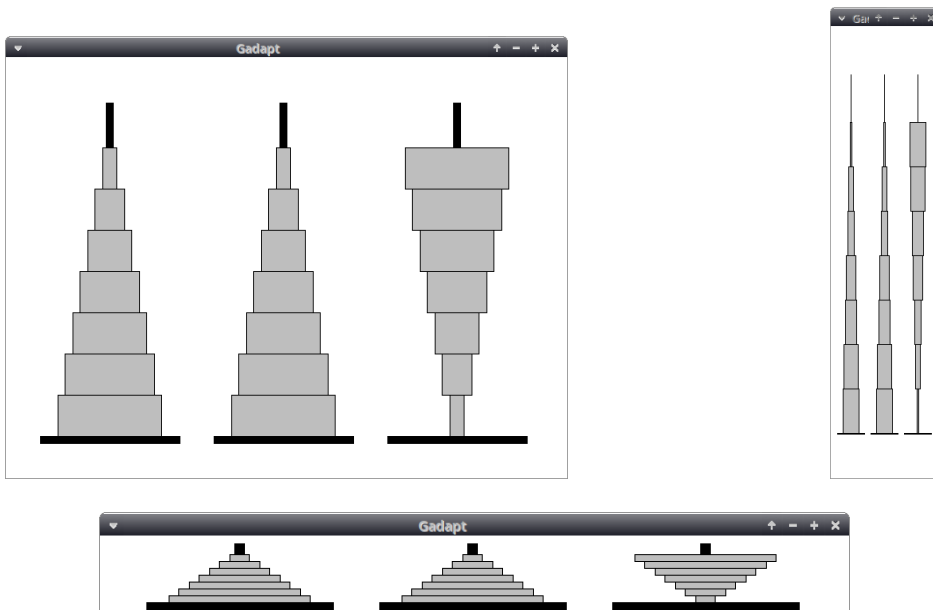


Abbildung 3: Beispiele für gefüllte Stapel bei unterschiedlichen Bildschirmfenstergeometrien. Der rechte Stapel wurde zu Testzwecken entgegen der Regeln für „Türme von Hanoi“ umgekehrt gefüllt.

4 Der Algorithmus für Türme von Hanoi

Zur Verschiebung des Turms von der linken Basis auf die rechte Basis darf, wie eingangs erwähnt, immer nur eine Scheibe pro Zug bewegt werden und niemals eine Scheibe auf einer kleineren zu liegen kommen. Ziel ist es, eine Funktion zu schreiben, welche diese Aufgabe leistet. Wir nennen diese Funktion `moveTower`.

Die Idee zur Lösung sieht so aus, dass man zur Verschiebung der untersten, größten Scheibe auf die rechte Basis offensichtlich vorher den restlichen Scheibenstapel auf die mittlere Basis verschieben muss.

Dieses Problem, einen Stapel auf eine andere Basis zu verschieben, kennen wir aber bereits. Dafür werden wir ja bald die Funktion `moveTower` geschrieben haben (Futur II!). Der einzige Unterschied ist jetzt, dass der zu verschiebende Scheibenstapel nun um eine Scheibe weniger hoch ist und die Zielbasis eine andere (nämlich die mittlere) ist.

Daher benötigt die Funktion `moveTower` folgende Parameter:

- **Array mit den Stapeldefinitionen:** Damit die Funktion auf die Türme zugreifen und die Scheiben verschieben kann, muss ihr das Array mit den Informationen zu den drei Stapeln übergeben werden, das in Abschnitt 3.1 angelegt wurde.
- **Ausgangsbasis:** Von welcher Basis soll ein Stapel wegbewegt werden?
- **Anzahl der zu bewegendenden Scheiben:** Wie im Beispiel eben gesehen, müssen nicht immer alle Scheiben eines Stapels verschoben werden, sondern nur ein Teil davon. Daher benötigt die Funkti-



Schreiben Sie die Funktion `moveTower` zur Verschiebung eines Scheibenstapels.

on einen Parameter zur Auswahl der Anzahl der zu verschiebenden Scheiben.

- **Zielbasis:** Auf welcher Basis soll sich der Scheibenstapel nach Abschluss der Funktion befinden?
- **Zwischenlager:** Auf welcher Basis können Scheiben zwischengelagert werden? Dieser Parameter ist nicht unbedingt nötig, denn es handelt sich dabei immer um diejenige Basis, die weder die Ausgangsbasis noch die Zielbasis ist. Aber die Angabe als Parameter erspart uns das aufwendige Suchen nach der „anderen“ Basis.

Der Algorithmus der Funktion sieht dann folgendermaßen aus:

- Falls der Turm keine Scheibe besitzt: Beende die Funktion.
- Bewege den störenden Turmstapel mit (n-1) Scheiben auf das Zwischenlager: Um die unterste Scheibe auf die Zielbasis schieben zu können, muss der restliche Stapel an Scheiben (also alle bis auf eine) auf das Zwischenlager verschoben werden. Rufen Sie dafür die Funktion `moveTower` mit den passenden Parametern auf. Überlegen Sie sich dabei insbesondere, welches Zwischenlager Sie für diese Operation angeben können.

Hinweis: Fertigen Sie eine Skizze an, um sich die Sachverhalte klar zu machen!

Frage: Klären Sie beim Überlegen die Punkte: Was ist die Ausgangsbasis? Was ist die Zielbasis? Wie viele Scheiben sind zu verschieben? Wo kann man zwischenlagern?

- Verschiebe die unterste Scheibe auf die Zielbasis: Nachdem der störende restliche Stapel sich nun im Zwischenlager befindet, kann die Scheibe einfach auf die Zielbasis verlagert werden.

Vergessen Sie dabei nicht, die im 0-ten Element gespeicherte Scheibenanzahl zu aktualisieren!

- Bewege den restlichen Stapel aus dem Zwischenlager auf die nun im Zielbereich befindliche unterste Scheibe: Hierfür kommt wieder ein Aufruf von `moveTower` zum Einsatz, wobei natürlich wieder die Parameter geeignet anzupassen sind.

Frage: Klären Sie beim Überlegen die Punkte: Was ist die Ausgangsbasis? Was ist die Zielbasis? Wie viele Scheiben sind zu verschieben? Wo kann man zwischenlagern?

Führen Sie die Funktion zur Verlagerung des gesamten Stapels von der linken Basis auf die rechte Basis in der Funktion `VtlKeyHit` aus. Sie wird dann gestartet, wenn Sie nach dem Start Ihres Programms eine beliebige Taste drücken.

Testen Sie nun Ihr Ergebnis!

Sie werden feststellen, dass das Programm perfekt funktioniert (wenn Sie alles richtig gemacht haben) und der Stapel nach dem Tastendruck (ggf. nach kurzem Nachdenken) auf dem rechten Stapel dargestellt wird.

Leider ist es etwas unbefriedigend, dass man dem Rechner beim Ausführen der Lösungsschritte nicht zusehen kann. Dies liegt daran, dass das Bild erst dann neu gezeichnet wird, wenn die Verlagerung des Scheibenstapels abgeschlossen ist.

Um diesem Missstand abzuhelpen, gehen Sie bitte folgendermaßen vor:

- Starten Sie die Funktion `moveTower` in `VtlKeyHit` nicht wie bisher direkt, sondern schreiben Sie eine neue kleine Funktion ohne Parameter oder Rückgabewert in der Sie den Aufruf für `moveTower` mit den richtigen Parametern platzieren.
- Starten Sie diese neue Funktion in `VtlKeyHit`, indem Sie sie der Funktion `runAsThread` übergeben.

Damit wird der Verlagerungsalgorithmus in einen parallel ausgeführten Thread ausgelagert und damit nebenläufig zur Bildschirmdarstellung ausgeführt.

- Damit der Algorithmus nicht zu schnell abläuft und auch etwas auf dem Bildschirm zu erkennen ist, fügen Sie bitte in der Funktion `moveTower` nach dem Verschieben der untersten Scheibe einen Aufruf der Funktion `delay` ein, wobei Sie den Verzögerungswert in Sekunden als Parameter angeben müssen.

Gute Verzögerungswerte können dabei zum Beispiel im Intervall $[0,05 \dots 0,5]$ Sekunden liegen. Experimentieren Sie!

5 Aufwand des Problems

Wenn Sie etwas mit der Scheibenanzahl experimentieren, stellen Sie schnell fest, dass die Anzahl der erforderlichen Züge mit der Scheibenanzahl sehr schnell wächst.

Um diese Intuition mit Fakten zu untermauern, ergänzen Sie Ihr Programm um eine Erfassung der Anzahl der durchgeführten Züge:

- Fügen Sie der Funktion `moveTower` einen Rückgabewert vom Typ `int` hinzu.
- Wenn die Höhe des zu verschiebenden Stapels 0 beträgt, geben Sie den Wert 0 zurück.
- Ansonsten legen Sie eine lokale Variable des Typs `int` an und initialisieren sie mit dem Wert 0.
- Auf diese Variable addieren Sie:



Fügen Sie eine Messung für den Aufwand der Turmverschiebung hinzu.

- Den Rückgabewert des ersten Aufrufs von `moveTower` zum Wegbewegen des Reststapels
- Die Zahl 1 für das Verschieben der untersten Scheibe in den Zielbereich
- Den Rückgabewert des zweiten Aufrufs von `moveTower` zum Wegbewegen des Reststapels
- Am Ende der Funktion geben Sie den Wert dieser Variable als Funktionswert zurück.

Damit Sie das Endergebnis sehen können, ergänzen Sie Ihre „neue kleine Funktion ohne Parameter oder Rückgabewert“ aus dem vorangehenden Abschnitt so, dass der Rückgabewert des allerersten Aufrufs der Funktion `moveTower` auf der Konsole ausgegeben wird.

Erstellen Sie eine Tabelle der getesteten Stapelhöhen zusammen mit den gemessenen benötigten Zugzahlen. Die Tabelle soll folgendes Aussehen haben:

n	Züge
1	1
2	...
3	...
4	...

Versuchen Sie möglichst große Werte, ohne Ihren Rechner oder die zur Verfügung stehende Zeit zu überfordern.

Hinweis: Entfernen Sie für dieses Experiment den Aufruf der Funktion `delay` aus Ihrem Programm um unnötige Verzögerungen zu vermeiden.

Sie können nun mit diesen Daten ein Diagramm erzeugen, zum Beispiel, indem Sie die Tabelle von Gnuplot [5] darstellen lassen, das Sie in den vorangehenden Aufgaben bereits kennengelernt haben.

Dadurch erhalten Sie einen guten Eindruck, wie der Rechenaufwand mit der Scheibenzahl wächst.



Erzeugen Sie aus den ermittelten Daten ein Diagramm.

Literatur

- [1] Wikipedia: Rekursion
<https://de.wikipedia.org/wiki/Rekursion>.
- [2] Wikipedia: Tower of Hanoi (der deutschsprachige Artikel weist derzeit gravierende Mängel auf)
https://en.wikipedia.org/wiki/Tower_of_Hanoi.
- [3] Wikipedia: Fraktal <https://de.wikipedia.org/wiki/Fraktal>.
- [4] Franz Kneißl, Michael Niemetz: *Rahmenprogramm WinAdapt für C++ Programme im Grafik-Mode*, OTH Regensburg, Juni 2022.
- [5] Gnuplot, <http://www.gnuplot.info/>.